

Clínica Unión Médica - Plataforma Digital Moderna

Este es el repositorio oficial de la nueva plataforma digital de la **Clínica Unión Médica**, desarrollada con tecnología de vanguardia para ofrecer una experiencia de usuario premium, rápida y accesible. El proyecto integra un portal público para pacientes y un panel de administración para la gestión interna.

Stack Tecnológico

El proyecto está construido sobre un stack moderno y eficiente:

- **Framework:** [Next.js 16](https://nextjs.org/) (<https://nextjs.org/>) (App Router)
- **Lenguaje:** [TypeScript](https://www.typescriptlang.org/) (<https://www.typescriptlang.org/>)
- **Estilos:** [Tailwind CSS 4](https://tailwindcss.com/) (<https://tailwindcss.com/>)
- **Base de Datos:** [SQLite](https://sqlite.org/) (<https://sqlite.org/>) (gestionado vía [Prisma ORM](https://www.prisma.io/) (<https://www.prisma.io/>))
- **IA Generativa:** Google Gemini 3 Flash (vía SDK oficial)
- **Renderizado:** React 19

Características Principales

Portal del Paciente (Público)

- **Diseño Glassmorphism:** Interfaz moderna con efectos de desenfoque y micro-interacciones fluidas.
- **Mega Menú Inteligente:** Navegación intuitiva organizada por departamentos y servicios.
- **Asistente Virtual (IA):** Chatbot "UMN" potenciado por Gemini para responder consultas, horarios y guiar a los pacientes 24/7.
- **Directorio Médico:** Búsqueda y filtrado de especialistas.
- **Servicios e Investigación:** Secciones detalladas sobre los servicios clínicos y publicaciones científicas.

Panel de Administración (Privado)

- **Ruta:** `/admin`
- **Dashboard:** Vista general del estado de la plataforma.
- **Gestión de Contenido:** Herramientas para actualizar médicos, servicios y noticias sin tocar código.
- **Autenticación:** Sistema de acceso para personal autorizado.

Arquitectura del Proyecto

El proyecto sigue una estructura modular basada en el **App Router** de Next.js (`/app`):

Directorios Clave (`/app`)

- `/` (**root**): Landing page y rutas públicas.
- `/admin`: Panel de control administrativo (protegido).
- `/login`: Portal de acceso para administradores.
- `/medicos`: Catálogo y perfiles de especialistas.
- `/servicios`: Información detallada de servicios médicos.
- `/nosotros`: Información institucional.
- `/api`: Endpoints de backend (Chatbot, búsquedas, etc.).

Componentes y Utilidades

- `/components`: Elementos de UI reutilizables (Botones, Inputs, Modales).
- `/lib`: Utilidades y configuraciones (Prisma Client, Helpers).
- `/prisma`: Esquema de base de datos (`schema.prisma`) y migraciones.

Configuración e Instalación

Sigue estos pasos para ejecutar el proyecto localmente:

1. Clonar el repositorio

```
git clone https://github.com/lifeshoftcode/union-medica.git
cd union-medica
```

2. Instalar dependencias

```
npm install
```

3. Configurar Variables de Entorno

Crea un archivo `.env` en la raíz del proyecto basado en el siguiente ejemplo:

```
# Clave API para el asistente Gemini
GEMINI_API_KEY=tu_clave_api_aqui

# URL de la base de datos (SQLite por defecto)
DATABASE_URL="file:./dev.db"
```

4. Inicializar Base de Datos

Genera el cliente de Prisma y prepara la base de datos local:

```
npx prisma generate
npx prisma db push
```

(Opcional) Si existe un `script de seed`, puedes correrlo para poblar datos iniciales:

```
npx prisma db seed
```

5. Iniciar Servidor de Desarrollo

```
npm run dev
```

La aplicación estará disponible en `http://localhost:3000`.

Inteligencia Artificial y Chatbot (UMN)

El asistente virtual utiliza una arquitectura **RAG (Retrieval-Augmented Generation)** híbrida para ofrecer respuestas precisas y contextualizadas.

Tecnologías y Técnicas

- **Motor de IA:** Google Gemini 3 Flash.
- **RAG Dinámico:** El sistema analiza la consulta del usuario, extrae palabras clave y busca en tiempo real en la base de datos SQLite (vía Prisma) para encontrar:
 - Médicos y especialistas relevantes.
 - Servicios clínicos.
 - Publicaciones científicas e investigaciones.
- **Base de Conocimiento Estática:** Se complementa con un módulo de conocimiento institucional (`knowledge.ts`) que contiene datos inmutables como historia, misión, ubicaciones y redes sociales.
- **Ingesta de Datos:** Incluye scripts (`extract_doctors.js`) para la migración y estructuración inicial de datos desde fuentes legacy.

Flujo de Respuesta

1. **User Query** -> 2. **Keyword Extraction** -> 3. **DB Search (Prisma)** -> 4. **Context Construction** -> 5. **Gemini Generation** -> 6. **Response**

Comandos Útiles

- `npm run dev`: Inicia el entorno de desarrollo.
- `npm run build`: Construye la aplicación para producción (incluye generación de Prisma).
- `npx prisma studio`: Abre una interfaz visual para gestionar la base de datos localmente.

Licencia

Este proyecto es privado y propiedad de Clínica Unión Médica.